Torneio de 8 Equipas: Estatísticas Individuais de Jogadores

Fase 1 · Modelação e núcleo funcional

Programação Funcional · Projeto em Haskell
Grupo: PLO3GR02 · Elementos: André Machado; Guilherme Silva; Tomás Monteiro.
Repositório: https://github.com/MCBaixinho/Programa-o-Olheiro

1. Descrição do projeto

Sistema em Haskell que regista as estatísticas individuais de cada jogador e as estatísticas coletivas de cada equipa em todos os jogos de um torneio de futebol com 8 equipas, e que no final permite perceber quem mais se destacou ao longo da competição.

Elemento	Descrição
Título	Torneio de 8 Equipas: Estatísticas Individuais de Jogadores
Problema	Num torneio com vários jogos é difícil comparar o contributo individual de cada jogador e o desempenho de cada equipa. O sistema regista o desempenho por partida e agrega-o para identificar os jogadores e equipas em destaque.
Objetivos	Modelar jogadores, equipas, jogos e estatísticas; calcular resultados, totais, médias e percentagens; produzir rankings e identificar o campeão e os jogadores em destaque do torneio.
Utilizadores	Treinadores, analistas e adeptos que queiram consultar o desempenho dos jogadores e das equipas.
Funcionalidades	Registo de estatísticas por jogo; resultado calculado a partir dos golos; totais por jogador; percentagem de passe e de remates enquadrados; melhor marcador e melhor assistente; cantos e foras de jogo por equipa; avanço de fase e campeão; relatório final.
Dados	Posicao, Jogador, Equipa, StatsPartida, StatsEquipa, Fase, Jogo, Torneio. 8 equipas e 7 jogos.
Entrada/Saída	Fase 1: dados definidos em código e resultados mostrados no terminal. Fase 3: dados lidos de ficheiros (CSV) e relatório final gravado em ficheiro.
Componente funcional	Tipos algébricos e records, pattern matching, recursão, folds, map/filter, composição, Maybe/Either, separação entre núcleo puro e IO.
Complexidade	Estrutura de torneio por fases (quartos, meias, final), muitas estatísticas derivadas por agregação, rankings e simulação de jogos com semente (extensão).
Equipa	[Nomes dos elementos do grupo]

2. Âmbito do torneio

O torneio tem exatamente 8 equipas e formato eliminatório:

```
Quartos de final (4 jogos) -> Meias-finais (2 jogos) -> Final (1 jogo)
```

- **7 jogos** no total: num torneio eliminatório o número de jogos é sempre o número de equipas menos um, porque cada jogo elimina uma equipa.
- O número de **equipas** em cada fase é par (8, 4, 2), o que permite emparelhá-las duas a duas sem sobrar nenhuma.
- **Dados de teste da Fase 1:** as 8 equipas com 3 jogadores cada (24 jogadores) e os 7 jogos completos, com 6 registos de estatísticas por jogo (42 registos). Na versão final as equipas terão plantéis completos.
- Como o formato é eliminatório, um jogo não pode terminar empatado. A regra de desempate fica definida para a Fase 2. Na Fase 1, a função `vencedor` devolve `Nothing` em caso de empate.

3. Modelo de domínio

A modelação antecede as funcionalidades. Os tipos ficam no módulo `Model.hs`:

```
-- | Model.hs
-- Tipos de dados do domínio: jogadores, equipas, jogos e estatísticas.
module Model where

-- | Posição em campo de um jogador.
data Posicao = GuardaRedes | Defesa | Medio | Avancado
  deriving (Show, Eq)

-- | Um jogador do torneio. O 'jId' identifica-o de forma única.
data Jogador = Jogador
  { jId      :: Int
  , jNome    :: String
  , jEquipa  :: String
  , jPosicao :: Posicao
  } deriving (Show, Eq)

-- | Uma equipa e o seu plantel.
data Equipa = Equipa
  { eNome      :: String
  , eJogadores :: [Jogador]
  } deriving (Show, Eq)

-- | Estatísticas individuais de UM jogador em UM jogo.
-- O jogador é referenciado pelo id (spJogador = jId).
data StatsPartida = StatsPartida
  { spJogador     :: Int
  , spMinutos     :: Int
  , spGolos       :: Int
  , spAssist      :: Int
  , spRemates     :: Int
  , spEnquadrados :: Int
  , spPassesTent  :: Int
  , spPassesComp  :: Int
  , spDesarmes    :: Int
  , spFaltas      :: Int
  , spAmarelos    :: Int
  , spVermelhos   :: Int
  } deriving (Show, Eq)

-- | Estatísticas coletivas de UMA equipa em UM jogo.
-- Cantos e foras de jogo são lances da equipa, não de um jogador.
data StatsEquipa = StatsEquipa
  { seCantos      :: Int
  , seForasDeJogo :: Int
  } deriving (Show, Eq)

-- | Fase do torneio (formato eliminatório).
data Fase = Quartos | Meias | Final
  deriving (Show, Eq, Ord, Enum, Bounded)

-- | Um jogo do torneio.
-- O resultado NÃO é guardado: calcula-se somando os golos dos jogadores
-- em 'jgStats', para evitar dados duplicados e inconsistentes.
data Jogo = Jogo
  { jgId        :: Int
  , jgFase      :: Fase
  , jgCasa      :: Equipa
  , jgFora      :: Equipa
  , jgStatsCasa :: StatsEquipa
  , jgStatsFora :: StatsEquipa
  , jgStats     :: [StatsPartida]
  } deriving (Show, Eq)

-- | O torneio completo: 8 equipas e os jogos disputados.
data Torneio = Torneio
  { tEquipas :: [Equipa]
  , tJogos   :: [Jogo]
  } deriving (Show, Eq)
```

3.1 Relações entre entidades

Relação	Forma de representação
Torneio tem 8 Equipas	`tEquipas :: [Equipa]`

Relação	Forma de representação
Torneio tem vários Jogos	`tJogos :: [Jogo]`
Equipa tem vários Jogadores	`eJogadores :: [Jogador]`
Jogo envolve 2 Equipas	Campos `jgCasa` e `jgFora`
Jogo tem estatísticas de cada equipa	Campos `jgStatsCasa` e `jgStatsFora` (cantos e foras de jogo)
Jogo tem várias StatsPartida	`jgStats :: [StatsPartida]`
StatsPartida pertence a um Jogador	Referência por identificador (`spJogador = jId`)

3.2 Decisões de modelação

- **Fase como tipo algébrico** em vez de String: o compilador impede fases inválidas e Enum/Bounded permitem percorrer as fases por ordem.
- **Posicao como tipo algébrico** permite pattern matching, por exemplo para tratar guarda-redes de forma diferente.
- **O resultado não é guardado:** os golos de cada equipa calculam-se somando os golos dos seus jogadores (`golosEquipa`). Assim é impossível existir um resultado diferente das estatísticas dos jogadores.
- **O jogo de cada registo não é repetido:** os registos `StatsPartida` estão dentro do `Jogo` a que pertencem, por isso não precisam de guardar o identificador do jogo.
- **Cantos e foras de jogo são estatísticas da equipa** e não de um jogador, por isso ficam num tipo próprio, `StatsEquipa`.
- **Estatísticas derivadas não são guardadas:** percentagens, totais, vencedores e rankings calculam-se com funções puras a partir dos registos.
- **StatsPartida referencia o jogador por id** para evitar copiar o jogador inteiro em cada registo.

4. Estrutura do projeto

```
torneio-stats/
├── README.md
├── cabal.project
├── torneio-stats.cabal
├── src/
│   ├── Model.hs
│   ├── Stats.hs
│   ├── Torneio.hs
│   └── DadosTeste.hs
├── app/
│   └── Main.hs
├── test/
├── data/
└── docs/
```

Módulo	Responsabilidade
`Model.hs`	Definição dos tipos de dados do domínio.
`Stats.hs`	Funções puras sobre estatísticas individuais: totais, contagens, percentagens, melhor marcador e melhor assistente.
`Torneio.hs`	Lógica do torneio: golos e vencedor de um jogo, emparelhamentos, jogos por fase, campeão, cantos e foras de jogo por equipa.
`DadosTeste.hs`	As 8 equipas e os 7 jogos do torneio para demonstração e testes em ghci.
`Main.hs`	Ponto de entrada: nesta fase, demonstração das funções no terminal.

As pastas `test/`, `data/` e `docs/` estão preparadas para as fases seguintes. Módulos previstos: `Ranking.hs`, `Validacao.hs`, `Parsing.hs` e `Simulacao.hs`.

5. Funções essenciais e recursão

Nesta fase privilegiam-se pattern matching e recursão explícita sobre listas. As funções de ordem superior (`map`, `filter`, folds) e a composição entram na Fase 2.

5.1 Módulo Stats

```
-- | Stats.hs
-- Funções puras sobre as estatísticas individuais dos jogadores.
-- Na Fase 1 usa-se apenas recursão explícita e pattern matching.
module Stats where

import Model

-- | Junta numa só lista os registos de estatísticas de todos os jogos.
todasStats :: [Jogo] -> [StatsPartida]
todasStats []     = []
todasStats (j:js) = jgStats j ++ todasStats js

-- | Total de golos de um jogador numa lista de registos.
totalGolos :: Int -> [StatsPartida] -> Int
totalGolos _ [] = 0
totalGolos i (s:ss)
  | spJogador s == i = spGolos s + totalGolos i ss
  | otherwise        = totalGolos i ss

-- | Total de assistências de um jogador.
totalAssist :: Int -> [StatsPartida] -> Int
totalAssist _ [] = 0
totalAssist i (s:ss)
  | spJogador s == i = spAssist s + totalAssist i ss
  | otherwise        = totalAssist i ss

-- | Total de minutos jogados por um jogador.
totalMinutos :: Int -> [StatsPartida] -> Int
totalMinutos _ [] = 0
totalMinutos i (s:ss)
  | spJogador s == i = spMinutos s + totalMinutos i ss
  | otherwise        = totalMinutos i ss

-- | Número de jogos em que o jogador tem registo.
contarJogos :: Int -> [StatsPartida] -> Int
contarJogos _ [] = 0
contarJogos i (s:ss)
  | spJogador s == i = 1 + contarJogos i ss
  | otherwise        = contarJogos i ss

-- | Passes (tentados, completos) de um jogador somados em todos os registos.
totalPasses :: Int -> [StatsPartida] -> (Int, Int)
totalPasses _ [] = (0, 0)
totalPasses i (s:ss)
  | spJogador s == i = (spPassesTent s + t, spPassesComp s + c)
  | otherwise        = (t, c)
  where (t, c) = totalPasses i ss

-- | Percentagem a partir de (parte, total). Devolve 0 se o total for 0.
percentagem :: Int -> Int -> Double
percentagem _ 0     = 0
percentagem parte total = 100 * fromIntegral parte / fromIntegral total

-- | Percentagem de passes completos num registo.
percentagemPasse :: StatsPartida -> Double
percentagemPasse s = percentagem (spPassesComp s) (spPassesTent s)

-- | Percentagem de remates enquadrados num registo.
percentagemEnquadrados :: StatsPartida -> Double
percentagemEnquadrados s = percentagem (spEnquadrados s) (spRemates s)

-- | Percentagem de passe de um jogador ao longo do torneio.
percentagemPasseTotal :: Int -> [StatsPartida] -> Double
percentagemPasseTotal i ss = percentagem comp tent
  where (tent, comp) = totalPasses i ss

-- | Procura um jogador pelo id.
procurarJogador :: Int -> [Jogador] -> Maybe Jogador
procurarJogador _ [] = Nothing
procurarJogador i (j:js)
```

```
  | jId j == i = Just j
  | otherwise  = procurarJogador i js

-- | Verifica se um jogador (pelo id) pertence a uma lista de jogadores.
pertence :: Int -> [Jogador] -> Bool
pertence _ []     = False
pertence i (j:js) = jId j == i || pertence i js

-- | Melhor marcador: jogador com mais golos (em empate fica o primeiro).
melhorMarcador :: [StatsPartida] -> [Jogador] -> Maybe (Jogador, Int)
melhorMarcador _ [] = Nothing
melhorMarcador ss (j:js) =
  case melhorMarcador ss js of
    Nothing -> Just (j, g)
    Just (j', g')
      | g >= g'   -> Just (j, g)
      | otherwise -> Just (j', g')
  where g = totalGolos (jId j) ss

-- | Melhor assistente: jogador com mais assistências.
melhorAssistente :: [StatsPartida] -> [Jogador] -> Maybe (Jogador, Int)
melhorAssistente _ [] = Nothing
melhorAssistente ss (j:js) =
  case melhorAssistente ss js of
    Nothing -> Just (j, a)
    Just (j', a')
      | a >= a'   -> Just (j, a)
      | otherwise -> Just (j', a')
  where a = totalAssist (jId j) ss

-- | Todos os jogadores de uma lista de equipas.
todosJogadores :: [Equipa] -> [Jogador]
todosJogadores []     = []
todosJogadores (e:es) = eJogadores e ++ todosJogadores es
```

5.2 Módulo Torneio

```
-- | Torneio.hs
-- Lógica do torneio: resultados, vencedores, fases e estatísticas de equipa
-- (cantos e foras de jogo).
module Torneio where

import Model
import Stats (pertence)

-- | Golos marcados por uma equipa num jogo, somando os golos dos seus
-- jogadores nos registos do jogo.
golosEquipa :: Equipa -> [StatsPartida] -> Int
golosEquipa _ [] = 0
golosEquipa e (s:ss)
  | pertence (spJogador s) (eJogadores e) = spGolos s + golosEquipa e ss
  | otherwise                             = golosEquipa e ss

-- | Golos da equipa da casa.
golosCasa :: Jogo -> Int
golosCasa j = golosEquipa (jgCasa j) (jgStats j)

-- | Golos da equipa de fora.
golosFora :: Jogo -> Int
golosFora j = golosEquipa (jgFora j) (jgStats j)

-- | Vencedor de um jogo; Nothing em caso de empate.
vencedor :: Jogo -> Maybe Equipa
vencedor j
  | golosCasa j > golosFora j = Just (jgCasa j)
  | golosFora j > golosCasa j = Just (jgFora j)
  | otherwise                 = Nothing

-- | Emparelha equipas duas a duas (8 equipas -> 4 jogos).
emparelhar :: [Equipa] -> [(Equipa, Equipa)]
emparelhar (a:b:resto) = (a, b) : emparelhar resto
emparelhar _           = []

-- | Jogos de uma determinada fase.
jogosDaFase :: Fase -> [Jogo] -> [Jogo]
jogosDaFase _ [] = []
jogosDaFase f (j:js)
  | jgFase j == f = j : jogosDaFase f js
  | otherwise     = jogosDaFase f js
```

```haskell
-- | Vencedores de uma lista de jogos (os empates são ignorados).
vencedoresDe :: [Jogo] -> [Equipa]
vencedoresDe [] = []
vencedoresDe (j:js) =
  case vencedor j of
    Just e  -> e : vencedoresDe js
    Nothing -> vencedoresDe js

-- | Fase seguinte, se existir.
faseSeguinte :: Fase -> Maybe Fase
faseSeguinte Quartos = Just Meias
faseSeguinte Meias   = Just Final
faseSeguinte Final   = Nothing

-- | Campeão: vencedor do (único) jogo da final, se existir.
campeao :: Torneio -> Maybe Equipa
campeao t =
  case jogosDaFase Final (tJogos t) of
    [final] -> vencedor final
    _       -> Nothing

-- | Total de cantos de uma equipa (pelo nome) em todos os jogos.
cantosEquipa :: String -> [Jogo] -> Int
cantosEquipa _ [] = 0
cantosEquipa n (j:js)
  | eNome (jgCasa j) == n = seCantos (jgStatsCasa j) + cantosEquipa n js
  | eNome (jgFora j) == n = seCantos (jgStatsFora j) + cantosEquipa n js
  | otherwise             = cantosEquipa n js

-- | Total de foras de jogo de uma equipa (pelo nome) em todos os jogos.
forasDeJogoEquipa :: String -> [Jogo] -> Int
forasDeJogoEquipa _ [] = 0
forasDeJogoEquipa n (j:js)
  | eNome (jgCasa j) == n = seForasDeJogo (jgStatsCasa j) + forasDeJogoEquipa n js
  | eNome (jgFora j) == n = seForasDeJogo (jgStatsFora j) + forasDeJogoEquipa n js
  | otherwise             = forasDeJogoEquipa n js

-- | Número de jogos disputados por uma equipa.
jogosDisputados :: String -> [Jogo] -> Int
jogosDisputados _ [] = 0
jogosDisputados n (j:js)
  | eNome (jgCasa j) == n || eNome (jgFora j) == n = 1 + jogosDisputados n js
  | otherwise                                      = jogosDisputados n js

-- | Total de cantos no torneio (as duas equipas, todos os jogos).
totalCantos :: [Jogo] -> Int
totalCantos []     = 0
totalCantos (j:js) = seCantos (jgStatsCasa j) + seCantos (jgStatsFora j) + totalCantos js
```

5.3 Resumo da recursão utilizada

Função	Padrão de recursão	Finalidade
`totalGolos`, `totalAssist`, `totalMinutos`	Acumulação sobre lista com guardas	Totais por jogador
`contarJogos`, `jogosDisputados`	Contagem com guardas	Jogos de um jogador / equipa
`totalPasses`	Acumulação de um par com `where`	Passes tentados e completos
`todasStats`, `todosJogadores`	Concatenação recursiva	Juntar registos / jogadores
`procurarJogador`	Pesquisa linear que devolve Maybe	Encontrar jogador por id
`pertence`	Pesquisa com `||`	Saber se um jogador é de uma equipa
`melhorMarcador`, `melhorAssistente`	Máximo por recursão com resultado intermédio	Destaques do torneio
`golosEquipa`	Acumulação com guarda que usa `pertence`	Golos de uma equipa num jogo
`emparelhar`	Pattern matching em dois elementos	Construir os jogos da fase seguinte
`jogosDaFase`	Filtragem recursiva	Selecionar jogos por fase
`vencedoresDe`	Recursão com case sobre Maybe	Equipas que avançam

Função	Padrão de recursão	Finalidade
`cantosEquipa`, `forasDeJogoEquipa`	Acumulação com várias guardas (casa / fora)	Estatísticas coletivas
`totalCantos`	Acumulação simples	Cantos de todo o torneio

6. Dados de teste

6.1 As 8 equipas e os jogos

Fase	Jogo	Resultado
Quartos	1	Porto 2-1 Benfica
Quartos	2	Sporting 3-0 Resende
Quartos	3	Amarante 1-2 Baião
Quartos	4	Braga 2-0 Penafiel
Meias	5	Porto 1-2 Sporting
Meias	6	Baião 1-0 Braga
Final	7	Sporting 1-2 Baião

Cada equipa tem 3 jogadores (avançado, médio e defesa), com ids de 1 a 24. Os nomes são fictícios. Os resultados acima não estão guardados em lado nenhum: são calculados pelo programa a partir dos golos dos jogadores.

6.2 Exemplo de um jogo em Haskell (DadosTeste.hs)

```
-- Ordem dos campos em StatsPartida:
--   jogador minutos golos assist remates enquadrados
--   passesTent passesComp desarmes faltas amarelos vermelhos
-- StatsEquipa: cantos foras_de_jogo

-- Porto 2-1 Benfica
jogo1 :: Jogo
jogo1 = Jogo 1 Quartos porto benfica (StatsEquipa 6 2) (StatsEquipa 4 3)
  [ StatsPartida 1 90 2 0 5 3 20 15 0 1 0 0
  , StatsPartida 2 90 0 2 1 0 50 45 3 2 1 0
  , StatsPartida 3 90 0 0 0 0 35 30 6 2 0 0
  , StatsPartida 4 90 1 0 4 2 15  9 1 0 0 0
  , StatsPartida 5 90 0 1 2 1 48 40 2 1 0 0
  , StatsPartida 6 90 0 0 0 0 30 24 5 3 1 0 ]
```

7. Demonstração

Compilação e execução, na pasta do projeto:

```
cabal build
cabal run
```

Saída do programa:

```
=== Torneio de 8 Equipas: Estatísticas (Fase 1) ===

--- Resultados ---
Quartos: Porto 2-1 Benfica
Quartos: Sporting 3-0 Resende
Quartos: Amarante 1-2 Baião
Quartos: Braga 2-0 Penafiel
Meias: Porto 1-2 Sporting
Meias: Baião 1-0 Braga
Final: Sporting 1-2 Baião

--- Quem passou dos quartos ---
Porto, Sporting, Baião, Braga

--- Emparelhamento das meias (a partir dos vencedores) ---
```

```
Porto vs Sporting
Baião vs Braga

--- Campeão ---
Baião

--- Destaques individuais ---
Melhor marcador:   Diogo Neves (Sporting) - 4 golos
Melhor assistente: Miguel Santos (Porto) - 3 assistências

--- Ficha de um jogador (id 7) ---
Nome:         Diogo Neves (Sporting, Avancado)
Jogos:        3
Minutos:      270
Golos:        4
Assistências: 1
Passe:        76.2%

--- Cantos e foras de jogo por equipa ---
Porto: 2 jogos, 11 cantos, 4 foras de jogo
Benfica: 1 jogo, 4 cantos, 3 foras de jogo
Sporting: 3 jogos, 19 cantos, 5 foras de jogo
Resende: 1 jogo, 2 cantos, 4 foras de jogo
Amarante: 1 jogo, 5 cantos, 2 foras de jogo
Baião: 3 jogos, 8 cantos, 6 foras de jogo
Braga: 2 jogos, 16 cantos, 3 foras de jogo
Penafiel: 1 jogo, 3 cantos, 3 foras de jogo
Total de cantos no torneio: 68
```

8. Evolução prevista

Fase	O que será acrescentado
Fase 2	Reescrever as funções repetitivas (ex.: `totalGolos`, `totalAssist`) com `map`, `filter`, folds e composição; rankings; validação dos dados com Maybe/Either (ex.: passes completos maiores que tentados); testes.
Fase 3	Leitura das equipas e jogos a partir de ficheiros CSV, menu no terminal e relatório final gravado em ficheiro.